Spring

Объекты, зависимости и контейнер

Spring Framework и Spring Boot

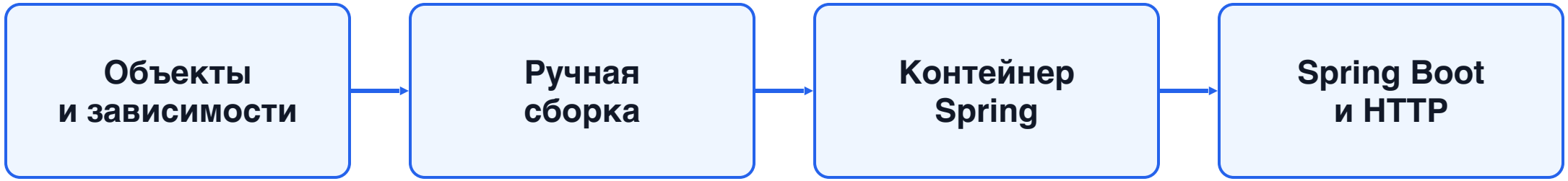


Что мы уже умеем

- Собирать Java-проект с Maven.
- Подключать библиотеку и работать с JSON.
- Читать HTTP-запрос и формировать ответ.
- Запускать сервер и обращаться к нему через curl.

Сегодня сохраним API и изменим устройство приложения.

Маршрут занятия



Каждый следующий шаг решает уже знакомую задачу.

Обычный Java-объект

Java • класс и фрагмент main

```
class TaskRepository {
    public int count() {
        return 3;
    }
}

var repository = new TaskRepository();
System.out.println(repository.count());
```

Класс описывает поведение.

new создаёт экземпляр.

Через ссылку вызываем метод.

Пока это программа без фреймворка.

Одному объекту нужен другой

Java • импорты опущены

```
class TaskService {
    private final TaskRepository repository =
        new TaskRepository();

    public int count() {
        return repository.count();
    }
}
```

Сервис использует хранилище.

Значит, хранилище является его зависимостью.

Сейчас сервис создаёт её сам.

Зависимость объекта здесь выражена ссылкой в поле.

Что связано внутри TaskService

- Сервис выполняет свою работу.
- Он также выбирает и создаёт хранилище.
- Чтобы передать другое хранилище, придётся менять класс.
- Общий экземпляр для нескольких сервисов передать неудобно.

Создание зависимости можно вынести за пределы класса.

Зависимость через конструктор

Java • импорты опущены

```
class TaskService {
    private final TaskRepository repository;

    public TaskService(TaskRepository repository) {
        this.repository = repository;
    }

    public int count() {
        return repository.count();
    }
}
```

Конструктор принимает ссылку.

Поле сохраняет её.

Метод работает с переданным объектом.

Объект получает зависимость снаружи. Это DI.

DI уже работает без Spring

Java • фрагмент main

```
var repository = new TaskRepository();
var service = new TaskService(repository);

System.out.println(service.count());
```

Создали хранилище.

Передали его в сервис.

Получили результат 3.

Для внедрения зависимости достаточно обычного конструктора.

Кто собирает приложение из объектов?

- Кто создаёт объекты?
- Кто знает порядок создания?
- Кто передаёт зависимости?
- А если таких объектов сто?

Java позволяет собрать всё вручную. Кто возьмёт эту работу на себя?

Ручная сборка приложения

Java • схема сборки внутри main

```
var repository = new TaskRepository();
var service = new TaskService(repository);
var controller = new TaskController(service);
```

Все связи видны в одном месте.

Порядок задаём сами.

Объекты получают готовые зависимости.

Это рабочий подход, особенно для небольшого приложения.

Когда сборка становится большой

- У объектов появляются собственные зависимости.
- Нужно выбрать реализации и передать настройки.
- Часть объектов должна жить всё время работы приложения.
- Некоторым ресурсам требуется корректное завершение.

Можно описать компоненты и поручить их сборку контейнеру.

Контейнер Spring

- Читает описание компонентов приложения.
- Создаёт управляемые объекты и передаёт зависимости.
- Хранит их согласно настройкам жизненного цикла.
- ApplicationContext даёт доступ к этой инфраструктуре.

В данном контексте контейнер работает внутри Java-процесса.

DI и IoC

Термин	Смысл в нашем примере
DI: Dependency Injection	Сервис получает хранилище через конструктор
IoC: Inversion of Control	Создание и связывание объектов поручаем контейнеру

DI можно использовать и при ручной сборке.

Бин: объект под управлением Spring

- Бином называют управляемый контейнером объект.
- TaskService может быть бином и оставаться обычным Java-классом.
- Контейнеру нужно сообщить, какие объекты создавать.
- Записи задач и DTO обычно создаёт прикладной код.

Класс описывает объект. Бин является экземпляром.

Явная регистрация через @Bean

Java • импорты опущены

```
@Configuration
class AppConfig {
    @Bean
    TaskRepository repository() {
        return new TaskRepository();
    }

    @Bean
    TaskService service(TaskRepository repository) {
        return new TaskService(repository);
    }
}
```

@Configuration отмечает конфигурацию.

@Bean описывает создание бина.

Параметр метода получает зависимость из контейнера.

Наши TaskRepository и TaskService можно оставить без аннотаций.

Контейнер без Spring Boot

Java • фрагмент main со Spring Context

```
try (var context =
        new AnnotationConfigApplicationContext(
            AppConfig.class)) {
    var service = context.getBean(TaskService.class);
    System.out.println(service.count());
}
```

Создаём контекст.

Он обрабатывает AppConfig.

Получаем собранный сервис.

Результат снова 3.

Spring умеет работать самостоятельно. Boot подключим дальше.

Аннотация: метаданные для инструмента

- Запись с @ добавляет сведения к классу, методу или параметру.
- Сама аннотация не запускает метод и не создаёт объект.
- Компилятор или библиотека должны обработать эти сведения.
- Spring читает свои аннотации при настройке приложения.

@Bean работает, когда конфигурацию обрабатывает Spring.

Поиск компонентов через аннотации

Java • отдельные файлы, метод сервиса опущен

```
@Repository
public class TaskRepository {
    public int count() { return 3; }
}

@Service
public class TaskService {
    private final TaskRepository repository;

    public TaskService(TaskRepository repository) {
        this.repository = repository;
    }
}
```

Spring может найти классы при сканировании пакетов.

@Service и @Repository обозначают роли компонентов.

Конструктор остаётся обычным.

Это альтернативная регистрация тех же компонентов.

Два способа описать бин

Способ	Когда удобно
@Component и его специализации	Наш класс можно отметить аннотацией
@Bean в конфигурации	Создание требует кода или класс взят из библиотеки

Оба способа регистрируют объекты в одном контейнере.

Почему используем конструктор

- Обязательные зависимости видны в сигнатуре.
- Ссылки можно сохранить в final-полях.
- Класс легко создать вручную для проверки.
- При единственном конструкторе @Autowired обычно не нужен.

В примерах курса зависимости передаём через конструктор.

Сколько экземпляров создаёт контейнер?

- По умолчанию у бина область видимости singleton.
- Один экземпляр приходится на определение бина в контейнере.
- Несколько компонентов могут получить ссылку на него.
- Новый HTTP-запрос обычно не создаёт новый контроллер.

Общий бин может обслуживать несколько запросов одновременно.

Если зависимости не сходятся

Ситуация	Что проверить
Подходящего бина нет	Регистрацию и область сканирования
Несколько подходящих бинов	Явный выбор: @Qualifier или @Primary
A нужен B, а B нужен A	Цикл зависимостей и разделение ответственности

Ошибка создания бина часто объясняет, какой связи не хватает.

Spring Framework, MVC и Boot

Название	Задача
Spring Framework	Контейнер и инфраструктура приложения
Spring MVC	HTTP-маршруты, параметры и ответы
Spring Boot	Автоконфигурация и удобный запуск Spring-приложения

Spring MVC входит в Spring Framework. Boot использует Spring.

Что добавляет Spring Boot

- Подключение согласованных наборов библиотек.
- Автоконфигурацию типовой инфраструктуры.
- Запуск веб-приложения со встроенным сервером.
- Настройки из файлов, окружения и аргументов запуска.

Код предметной задачи по-прежнему пишем сами.

Новый проект через Spring Initializr

Параметры учебного проекта

start.spring.io

Project: Maven
Language: Java
Spring Boot: стабильный релиз 4.x
Java: 21
Packaging: Jar
Dependencies: Spring Web

Получаем заготовку проекта.

pom.xml описывает сборку.

src/main/java содержит код.

В проекте есть Maven Wrapper.

Версию Boot фиксируем в pom.xml проекта.

Starter подключает набор зависимостей

pom.xml • фрагмент для Spring Boot 4.x

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-webmvc
  </artifactId>
</dependency>
```

Пример для Spring Boot 4.x.

Starter приводит нужные транзитивные зависимости.

Версии согласует конфигурация Boot.

Maven скачивает зависимости. Spring Boot настраивает приложение.

Как работает автоконфигурация

- Boot проверяет библиотеки в classpath.
- Учитывает свойства и уже описанные бины.
- При выполнении условий добавляет типовую конфигурацию.
- Часть настроек можно заменить своей конфигурацией.

Набор зависимостей влияет на поведение приложения при запуске.

Точка входа в приложение

Java • импорты опущены

```
@SpringBootApplication
public class TasksApplication {
    public static void main(String[] args) {
        SpringApplication.run(
            TasksApplication.class, args);
    }
}
```

main остаётся точкой входа Java.

run запускает Spring-приложение.

В нашем веб-проекте также стартует сервер.

Spring Boot запускается как обычная Java-программа.

Что включает @SpringBootApplication

Механизм	Для чего нужен
Конфигурация Spring Boot	Позволяет описывать бины приложения
Сканирование компонентов	Находит наши отмеченные классы
Автоконфигурация	Подключает инфраструктуру по условиям

Поиск наших компонентов и автоконфигурация решают разные задачи.

Расположение классов имеет значение

Схема пакетов • каждый класс в своём файле

```
package example.tasks;
// TasksApplication

package example.tasks.web;
// TaskController

package example.tasks.service;
// TaskService

package example.tasks.storage;
// TaskRepository
```

Каждый блок относится к отдельному файлу.

Класс приложения находится в общем родительском пакете.

Соседний пакет вне области сканирования можно случайно пропустить.

Настройки приложения

application.properties и отдельная команда терминала

```
server.port=8080
spring.application.name=tasks-api


java -jar target/tasks-api.jar --server.port=8081
```

Первые две строки:
application.properties

Нижняя строка:
пример запуска с другим портом.

Настройки окружения можно менять без правки Java-кода.

Первый HTTP-контроллер

Java • импорты опущены

```
@RestController
class HelloController {
    @GetMapping("/hello")
    public String hello() {
        return "Hello, HTTP!";
    }
}
```

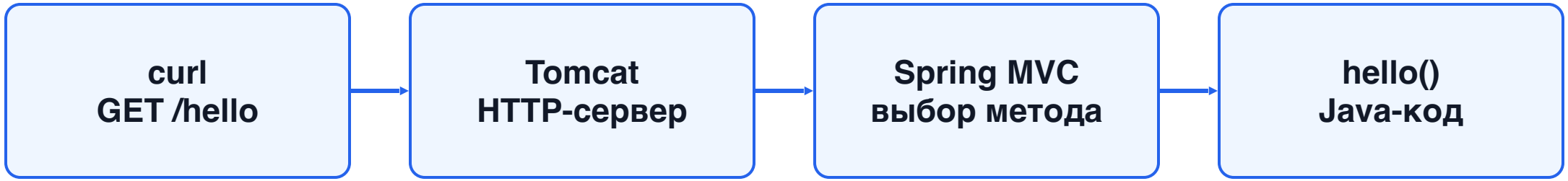
@RestController регистрирует веб-компонент.

@GetMapping связывает GET /hello с методом.

Строка станет телом ответа.

Метод вызывает Spring MVC при подходящем запросе.

Путь HTTP-запроса



Возвращённое значение MVC преобразует в HTTP-ответ.

Контроллер получает сервис

Java • импорты опущены

```
@RestController
class TaskController {
    private final TaskService service;

    TaskController(TaskService service) {
        this.service = service;
    }

    @GetMapping("/tasks/count")
    public int count() {
        return service.count();
    }
}
```

Знакомый конструктор с зависимостью.

Spring передаёт бин TaskService.

Метод контроллера вызывает сервис.

Контроллер и сервис связаны той же ссылкой, что при ручной сборке.

Java-объект в теле ответа

Java • импорты опущены

```
record Greeting(String message) {}

@RestController
class GreetingController {
    @GetMapping("/greeting")
    public Greeting greeting() {
        return new Greeting("Hello!");
    }
}

// Ответ: {"message":"Hello!"}
```

Greeting содержит данные.

Контроллер возвращает объект.

HTTP-конвертер с Jackson записывает JSON.

Объект ответа можно создать через new. Ему не нужен статус бина.

Как данные запроса попадают в метод

Аннотация	Источник	Пример запроса
@PathVariable	Часть пути	/tasks/42
@RequestParam	Параметр query	/tasks?done=false
@RequestBody	Тело запроса	JSON при POST /tasks

Связь HTTP и Java описываем явно в сигнатуре метода.

Пример параметра запроса

Java • замена метода в HelloController

```
@GetMapping("/hello")
public String hello(
        @RequestParam(name = "name",
                      defaultValue = "world")
        String name) {
    return "Hello, " + name + "!";
}

// GET /hello?name=Java
// Тело ответа: Hello, Java!
```

Параметр name берём из URL.

Если его нет, используем world.

Это новая версия метода hello().

Маршрут и параметры разбирает Spring MVC.

Запуск и проверка через curl

Терминал • команды выполняем по очереди

```
./mvnw spring-boot:run

// В другом терминале:
curl -i http://localhost:8080/hello

// После добавления параметра name:
curl -i "http://localhost:8080/hello?name=Java"
```

Maven Plugin запускает приложение.

Сервер продолжает работать в первом терминале.

curl отправляет запрос из второго.

Клиенту достаточно знать контракт HTTP.

Когда приложение не заработало

Симптом	Первая проверка
Не компилируется	Зависимости Maven, импорты и JDK
Ошибка создания бина	Тип зависимости и регистрация компонента
Порт занят	Другая копия сервера или server.port
Получили 404	Путь, сканирование и регистрация контроллера

Сначала определяем этап ошибки, затем читаем причину в логах.

Что проверить перед демонстрацией

- Можно ли внедрить зависимость без Spring?
- Что превращает объект в бин?
- Кто вызывает метод с @GetMapping?
- Чем Spring Boot помогает Spring-приложению?

Объяснение должно связывать аннотацию с конкретным действием.

Материалы к занятию

- Spring Framework: DI и контейнер.
- Java-конфигурация, сканирование и области видимости бинов.
- Spring Boot: первое приложение и автоконфигурация.
- Spring MVC: контроллеры и обработка запросов.

Ссылки на официальную документацию находятся в заметках.

Демонстрация: знакомый API на Spring Boot

Создание проекта и запуск /hello
Контроллер с зависимостью, JSON и проверка через curl